

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 2 – MICROARQUITETURA (AULA 14)

Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

Se você for esperto, em 9 segundos ...

Spot 3 differences

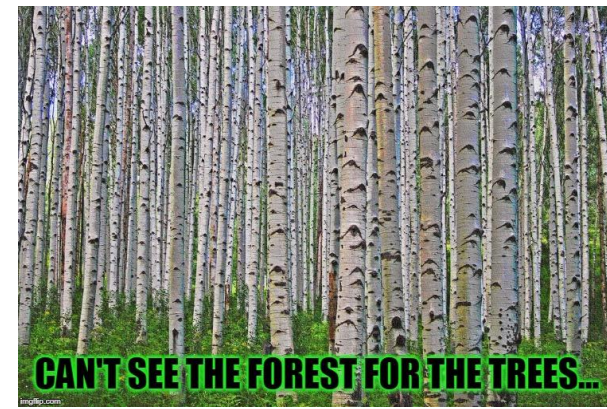


Tópico 2: A grande pergunta

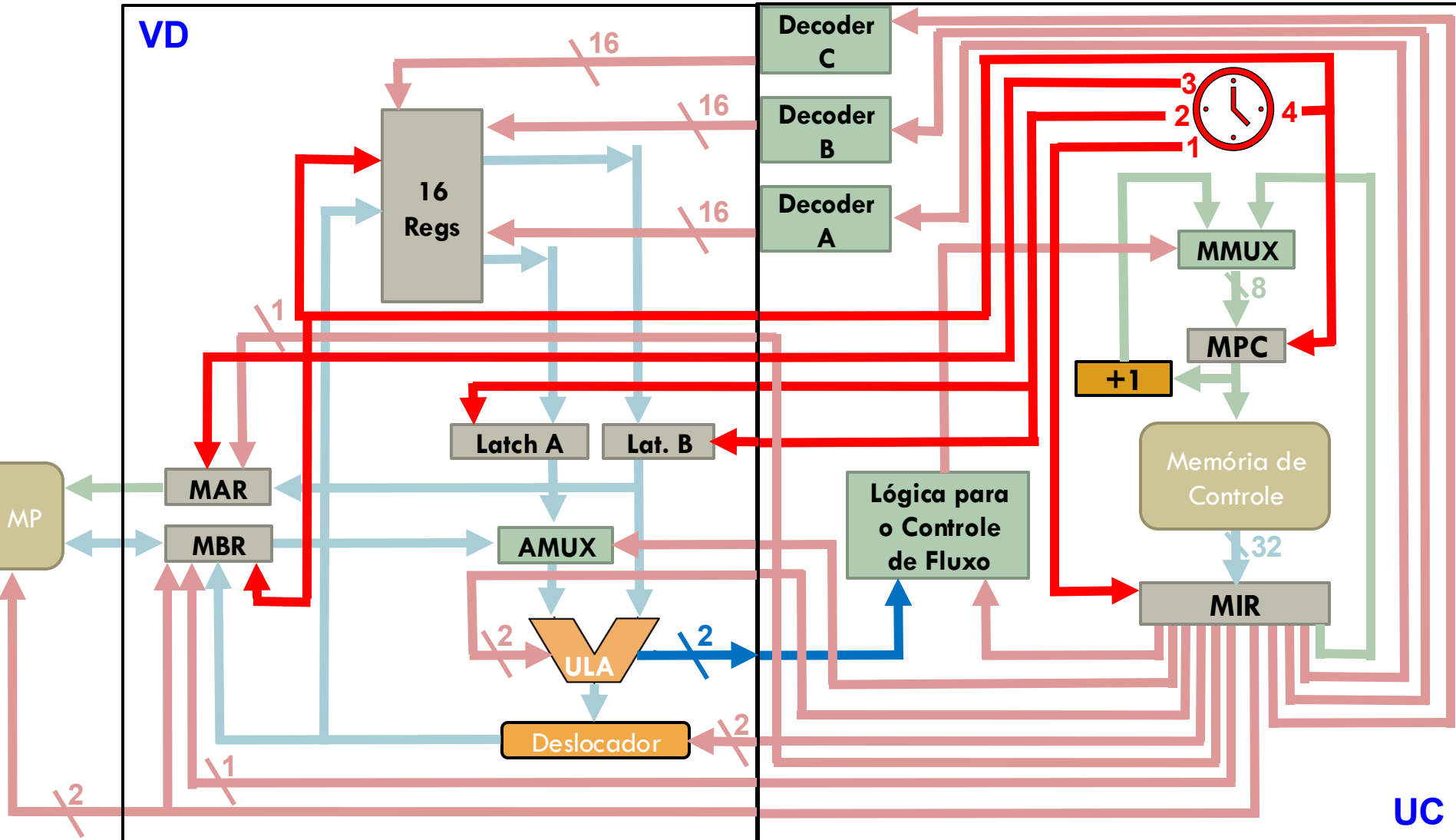


Exercício de Tópico 1

- Considere que uma instrução do nível n pode fazer o trabalho de b instruções do nível $n-1$, ou seja, uma instrução no nível n é b vezes mais poderosa do que uma instrução no nível $n-1$. Se um **programa** de nível 1 requer s segundos para ser executado, quanto tempo **programas equivalentes** levariam nos níveis 2 e 3, assumindo que p instruções no nível n são necessárias para interpretar uma única instrução no nível $n+1$?
- É uma pergunta simples, mas exemplifica a dificuldade que muitos tem
 - de ler e entender o que está sendo solicitado e
 - de não ver a floresta por causa das árvores!



O projeto do microprocessador MIC-1

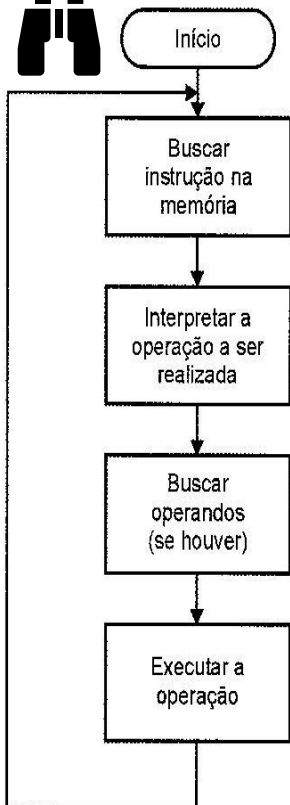


O Ciclo de VN para o Macroprograma

OPCODE

OPERANDOS

AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)



- O **μprograma** tem que interpretar o **macroprograma**, isto é, implementa o modelo VN do macroprograma
- Por ser um programa, o ciclo VN seria realizada em **software**
 - Busca a próxima **macroinstrução**
 - Identifica a macroinstrução buscada
 - Executa uma sequência de **microinstruções** equivalente a macroinst
 - Repete

Figura 2.6 Ciclo básico de instrução.

Nossa ISA

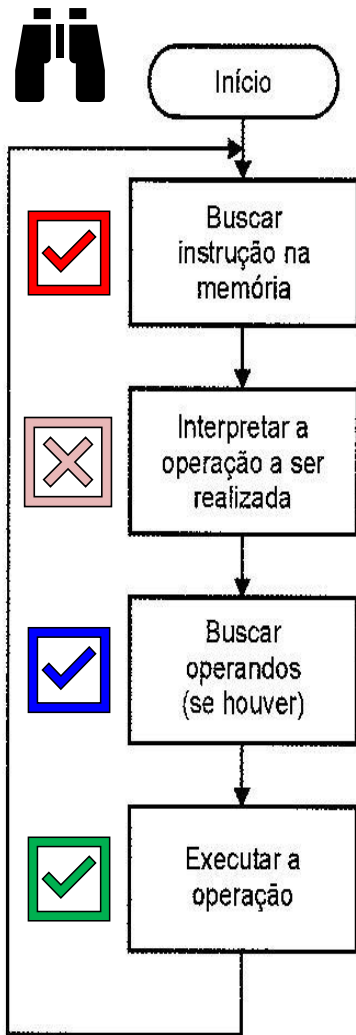
ISA MAC-1 (nível 2)

- 23 Macroinstruções
- de 16 bits (coluna 1)
- Opcode crescente (4 ou 7 bits), não de tamanho fixo
- Três tipos de endereçamento:
 - Endereçamento direto
 - Endereçamento imediato
 - Endereçamento indexado
- Um operando de 12 bits ou 8 bits ou sem operando
 - Não determina diretamente o número de dados necessário para a operação

	Binary	Mnemonic	Instruction	Meaning
0000	xxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001	xxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010	xxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011	xxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100	xxxxxxxxxxx	JPOS	Jump positive	If $ac \geq 0$ then $pc := x$
0101	xxxxxxxxxxx	JZER	Jump zero	If $ac = 0$ then $pc := x$
0110	xxxxxxxxxxx	JUMP	Jump	$pc := x$
0111	xxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000	xxxxxxxxxxx	LODL	Load local	$ac := m[sp + x]$
1001	xxxxxxxxxxx	STOL	Store local	$m[x + sp] := ac$
1010	xxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp + x]$
1011	xxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp + x]$
1100	xxxxxxxxxxx	JNEG	Jump negative	If $ac < 0$ then $pc := x$
1101	xxxxxxxxxxx	JNZE	Jump nonzero	If $ac \neq 0$ then $pc := x$
1110	xxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000	00000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001	00000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010	00000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011	00000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100	00000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101	00000000	SWAP	Swap ac, sp	$tmp := ac$; $ac := sp$; $sp := tmp$
1111110	yyyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
1111111	yyyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

xxxxxxxxxxx is a 12-bit machine address; in column 4 it is called x .
 yyyyyyy is an 8-bit constant; in column 4 it is called y .

O Microprograma de MIC-1



- O **microprograma** tem que interpretar o **macroprograma**
 - Busca a próxima macroinstrução
 - Identifica a macroinstrução buscada
 - Executa uma sequência de microinstruções equivalente
 - Repete
- Já temos visto como usar o Via de Dados
- A UC2 determina que acontece, e quando, durante num ciclo
- O **microprograma** define que acontece ciclo a ciclo
- A interpretação ou realização de uma macroinstrução leva vários ciclos (requer a execução de várias microinstruções)
- **Por quê?**

Figura 2.6 Ciclo básico de instrução.

Busca na memória de MIC-1

	OPCODE									OPERANDOS			
	AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)
0:	0	00	00	00	0	1	1	0	0	0000	0000	0000	(01) ₁₆

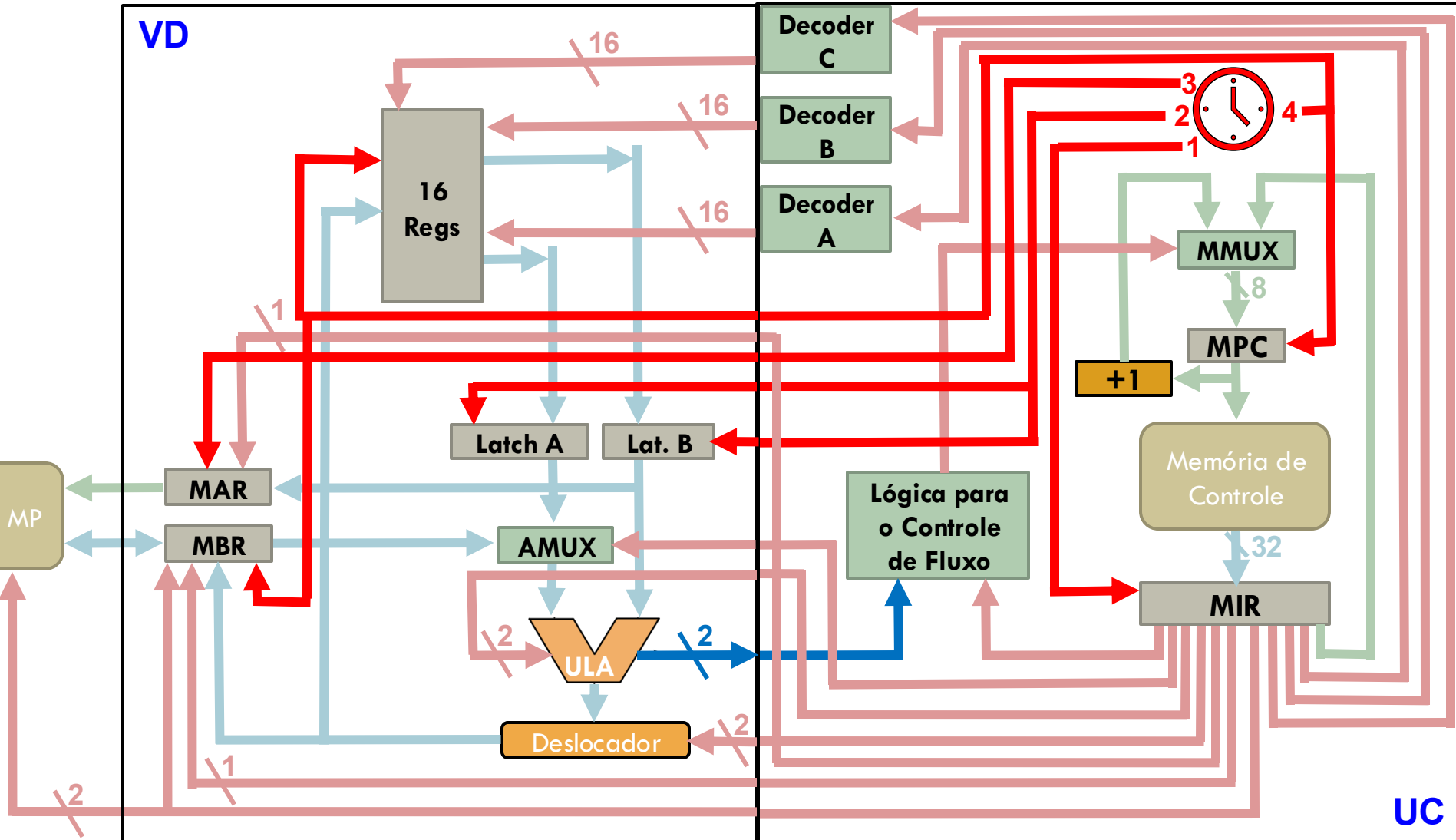


- Dado que a latência da "memória principal" é seis subciclos
- $IR = mp[PC];$
 - 0: $mar := pc; rd;$
 - 1: $pc := pc + 1; rd;$
 - 2: $ir := mbr;$

C	SC	Ação
1	1	Busca da $\mu inst$
1	2	Busca pc, rd=1
1	3	End = ??, rd=1
1	4	$mar := pc; rd=1$
2	1	Busca da $\mu inst$; rd=1
2	2	Busca pc, 1; rd=1
2	3	Soma; rd=1
2	4	Soma; rd=1
3	1	Busca da $\mu inst$; rd=1
3	2	Amux=1; EnC=1; rd=0

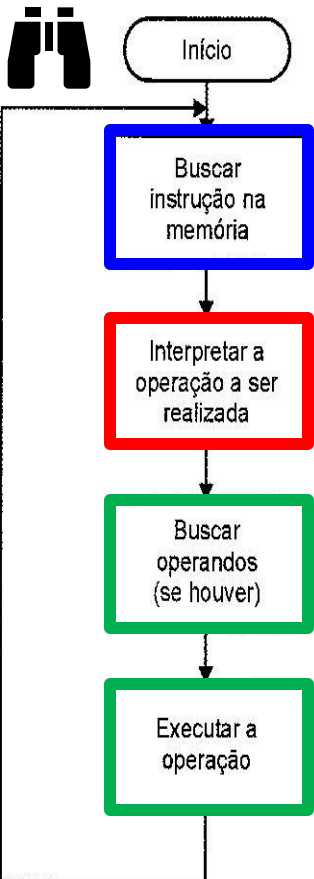
Figura 2.6 Ciclo básico de instrução.

O projeto do microprocessador MIC-1



Identificando a macroinstrução

OPCODE									OPERANDOS			
AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)
3: 0	01	00	10	0	0	0	0	1	0100	0011	0011	(13) ₁₆



□ Macroinstrução tem 16 bits:

- 2: $ir := mbr$; if n then goto 28;
- 3: $tir := lshift(ir + ir)$; if n then goto 19;
- 4: $tir := lshift(tir)$; if n then goto 11;
- 5: $alu := tir$; if n then goto 9;
- 6: $mar := ir$; rd;
- 7: rd;
- 8: $ac := mbr$; goto 0;
- 9: $mar := ir$; $mbr := ac$; wr;
- 10: wr; goto 0;

Interpreta

A instrução é negativo?

Executa {0000 = Lodd}

Executa

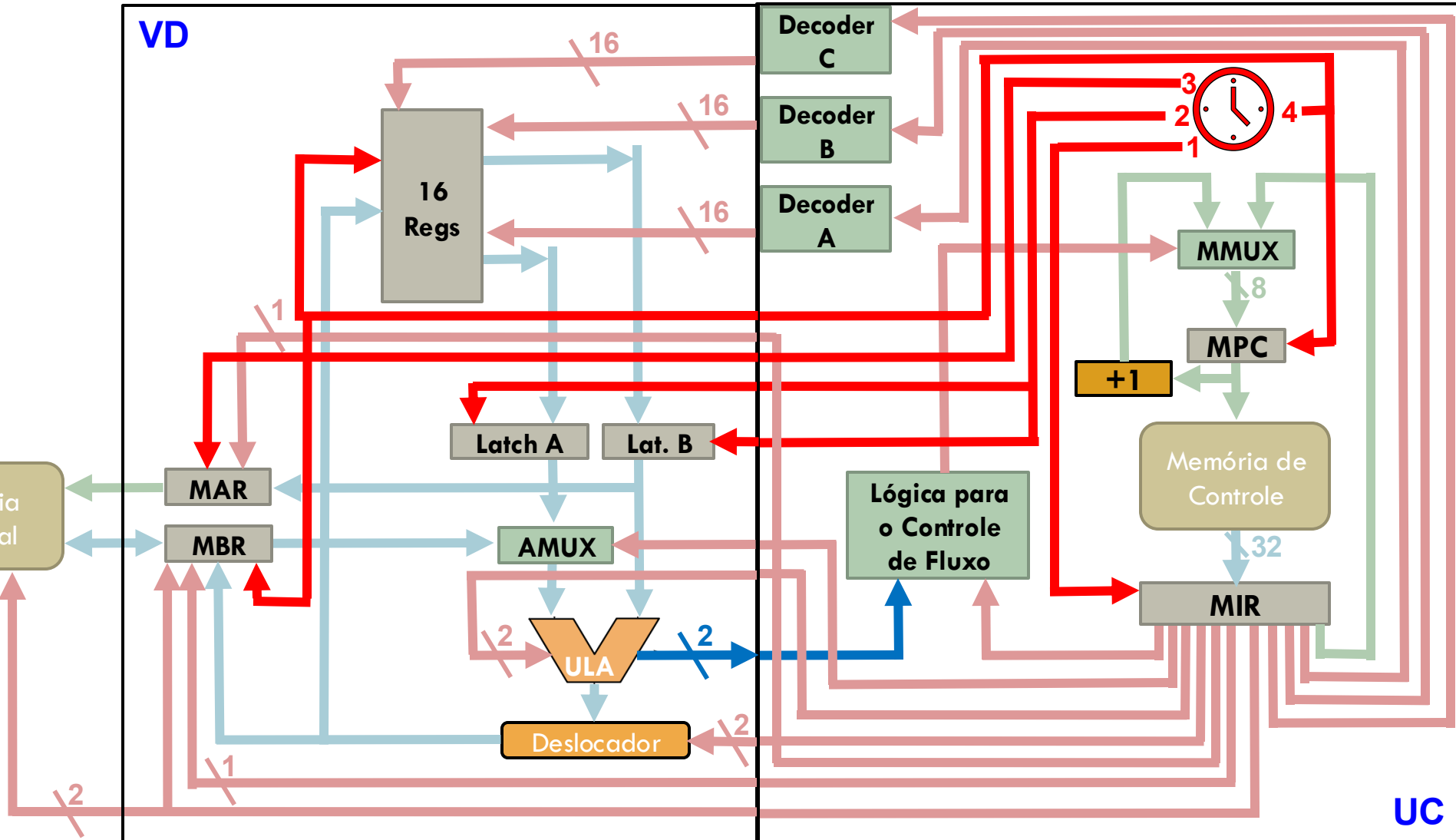
Executa

Executa {0001 = STOD}

Executa

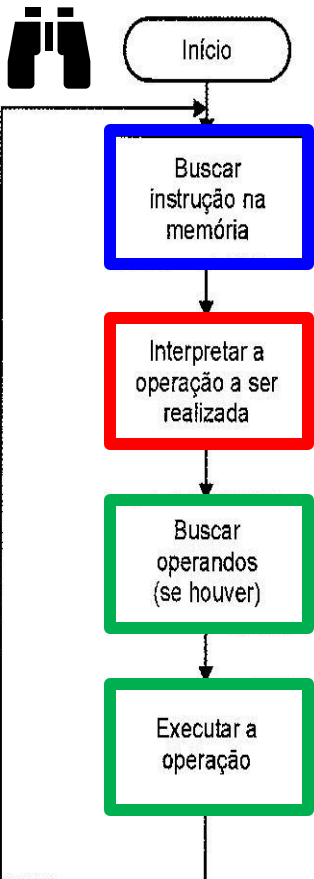
Figura 2.6 Ciclo básico de instrução.

O projeto do microprocessador MIC-1



Identificando a macroinstrução

OPCODE									OPERANDOS			
AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)
3: 0	01	00	10	0	0	0	0	1	0100	0011	0011	13



□ Macroinstrução tem 16 bits: ABCDEFGHIJKLMNOP

- 2: *ir* := *mbr*; if *n* then goto 28; *if A = 1 then goto 28*
- 3: *tir* := *lshift*(*ir* + *ir*); if *n* then goto 19; *if B = 1 then goto 19*
- 4: *tir* := *lshift*(*tir*); if *n* then goto 11; *if C = 1 then goto 11*
- 5: *alu* := *tir*; if *n* then goto 9; *if D = 1 then goto 9*
- 6: *mar* := *ir*; *rd*; Executa {0000 = LODD}
- 7: *rd*; Executa
- 8: *ac* := *mbr*; goto 0; Executa
- 9: *mar* := *ir*; *mbr* := *ac*; *wr*; Executa {0001 = STOD}
- 10: *wr*; goto 0; Executa

Figura 2.6 Ciclo básico de instrução.

Um Microprograma de MIC-1

- Aqui está um microprograma, de 79 microinstruções de 32 bits

0: mar:=pc; rd;	Busca	40: tir:=lshift(tir); if n then goto 46;	
1: pc:=pc + 1; rd;	Busca	41: alu:=tir; if n then goto 44;	
2: ir:=mbr; if n then goto 28;	Busca/Identifica	42: alu:=ac; if n then goto 22;	{1100 = JNEG}
3: tir:=lshift(ir + ir); if n then goto 19;	Identifica	43: goto 0;	
4: tir:=lshift(tir); if n then goto 11;	Identifica	44: alu:=ac; if z then goto 0;	{1101 = JNZE}
5: alu:=tir; if n then goto 9;	Identifica	45: pc:=band(ir.amask); goto 0;	
6: mar:=ir; rd;	Executa	46: tir:=lshift(tir); if n then goto 50;	
	{0000 = LODD}	47: sp:=sp + (-1);	{1110 = CALL}
7: rd;	Executa	48: mar:=sp; mbr:=pc; wr;	
8: ac:=mbr; goto 0;	Executa	49: pc:=band(ir.amask); wr; goto 0;	
9: mar:=ir; mbr:=ac; wr;	{0001 = STOD}	50: tir:=lshift(tir); if n then goto 65;	
10: wr; goto 0;	Executa	51: tir:=lshift(tir); if n then goto 59;	
11: alu:=tir; if n then goto 15;	Identifica	52: alu:=tir; if n then goto 56;	
12: mar:=ir; rd;	{0010 = ADDD}	53: mar:=ac; rd;	{1111-0000 = PSHI}
13: rd;	Executa	54: sp:=sp + (-1); rd;	
14: ac:=mbr + ac; goto 0;	Executa	55: mar:=sp; wr; goto 10;	
15: mar:=ir; rd;	{0011 = SUBD}	56: mar:=sp; sp:=sp + 1; rd;	{1111-0010 = POPI}
16: ac:=ac + 1; rd;	Executa	57: rd;	
17: a:=inv(mbr);	Executa	58: mar:=ac; wr; goto 10;	
18: ac:=ac + a; goto 0;	Executa	59: alu:=tir; if n then goto 62;	
19: tir:=lshift(tir); if n then goto 25;	Identifica	60: sp:=sp + (-1);	{1111-0100 = PUSH}
20: alu:=tir; if n then goto 23;		61: mar:=sp; mbr:=ac; wr; goto 10;	
21: alu:=ac; if n then goto 0;	{0100 = JPOS}	62: mar:=sp; sp:=sp + 1; rd;	{1111-0110 = POP}
22: pc:=band(ir.amask); goto 0;		63: rd;	
23: alu:=ac; if z then goto 22;	{0101 = JZER}	64: ac:=mbr; goto 0;	
24: goto 0;		65: tir:=lshift(tir); if n then goto 73;	
25: alu:=tir; if n then goto 27;		66: alu:=tir; if n then goto 70;	
26: pc:=band(ir.amask); goto 0;	{0110 = JUMP}	67: mar:=sp; sp:=sp + 1; rd;	{1111-1000 = RETN}
27: ac:=band(ir.amask); goto 0;	{0111 = LOCO}	68: rd;	
28: tir:=lshift(ir + ir); if n then goto 40;		69: pc:=mbr; goto 0;	
29: tir:=lshift(tir); if n then goto 35;		70: a:=ac;	{1111-1010 = SWAP}
30: alu:=tir; if n then goto 33;		71: ac:=sp;	
31: a:=ir + sp;	{1000 = LODL}	72: sp:=a; goto 0;	
32: mar:=a; rd; goto 7;		73: alu:=tir; if n then goto 76;	
33: a:=ir + sp;	{1001 = STOL}	74: a:=band(ir.smask);	{1111-1100 = INSP}
34: mar:=a; mbr:=ac; wr; goto 10;		75: sp:=sp + a; goto 0;	
35: alu:=tir; if n then goto 38;		76: a:=band(ir.smask);	{1111-1110 = DESP}
36: a:=ir + sp;	{1010 = ADDL}	77: a:=inv(a);	
37: mar:=a; rd; goto 13;		78: a:=a + 1; goto 75;	
38: a:=ir + sp;	{1011 = SUBL}		
39: mar:=a; rd; goto 16 ;			

Identificando a macroinstrução

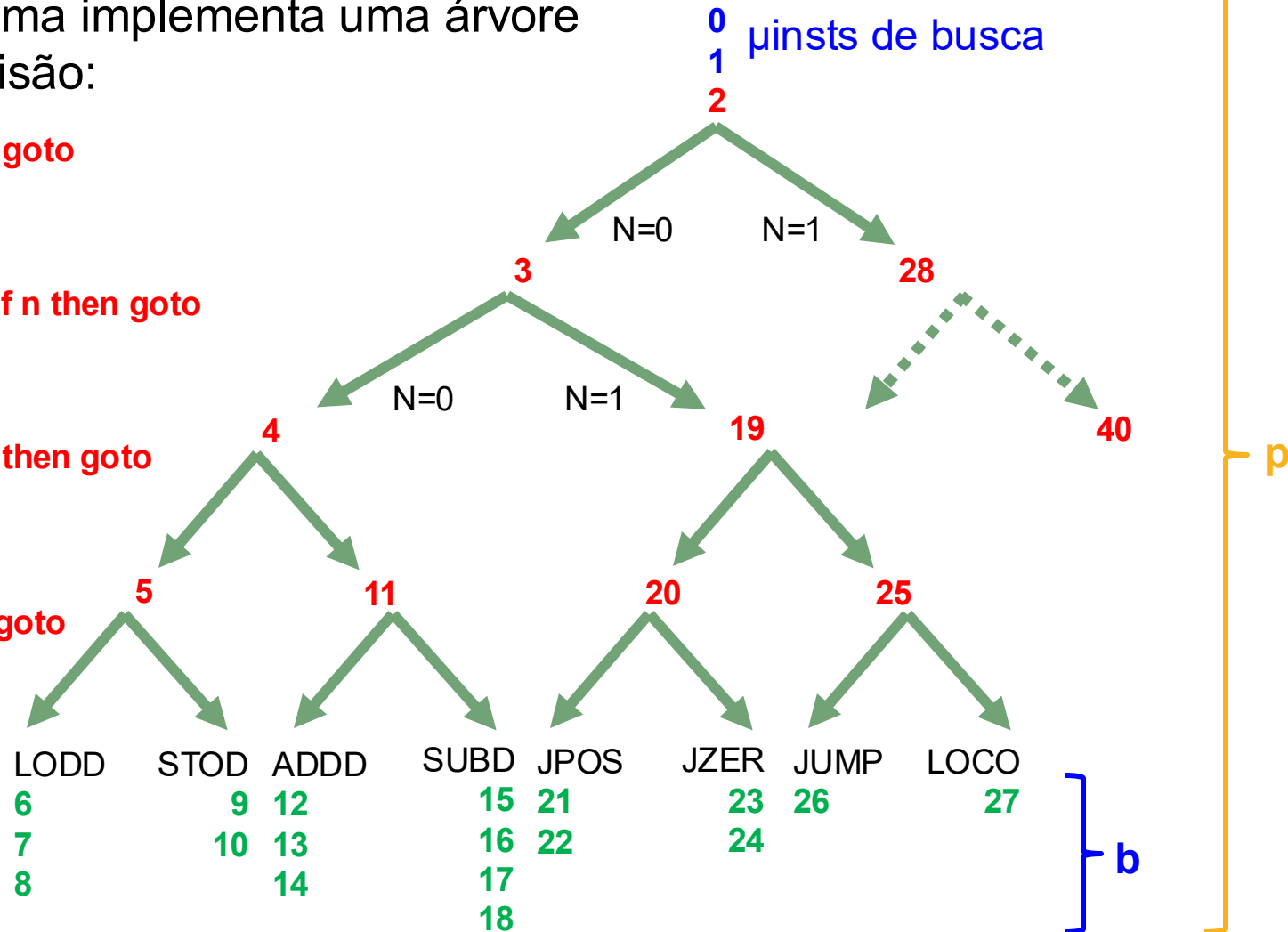
- O microprograma implementa uma árvore binária de decisão:

ir := mbr; if n then goto

tir := lshift(ir + ir); if n then goto

tir := lshift(tir); if n then goto

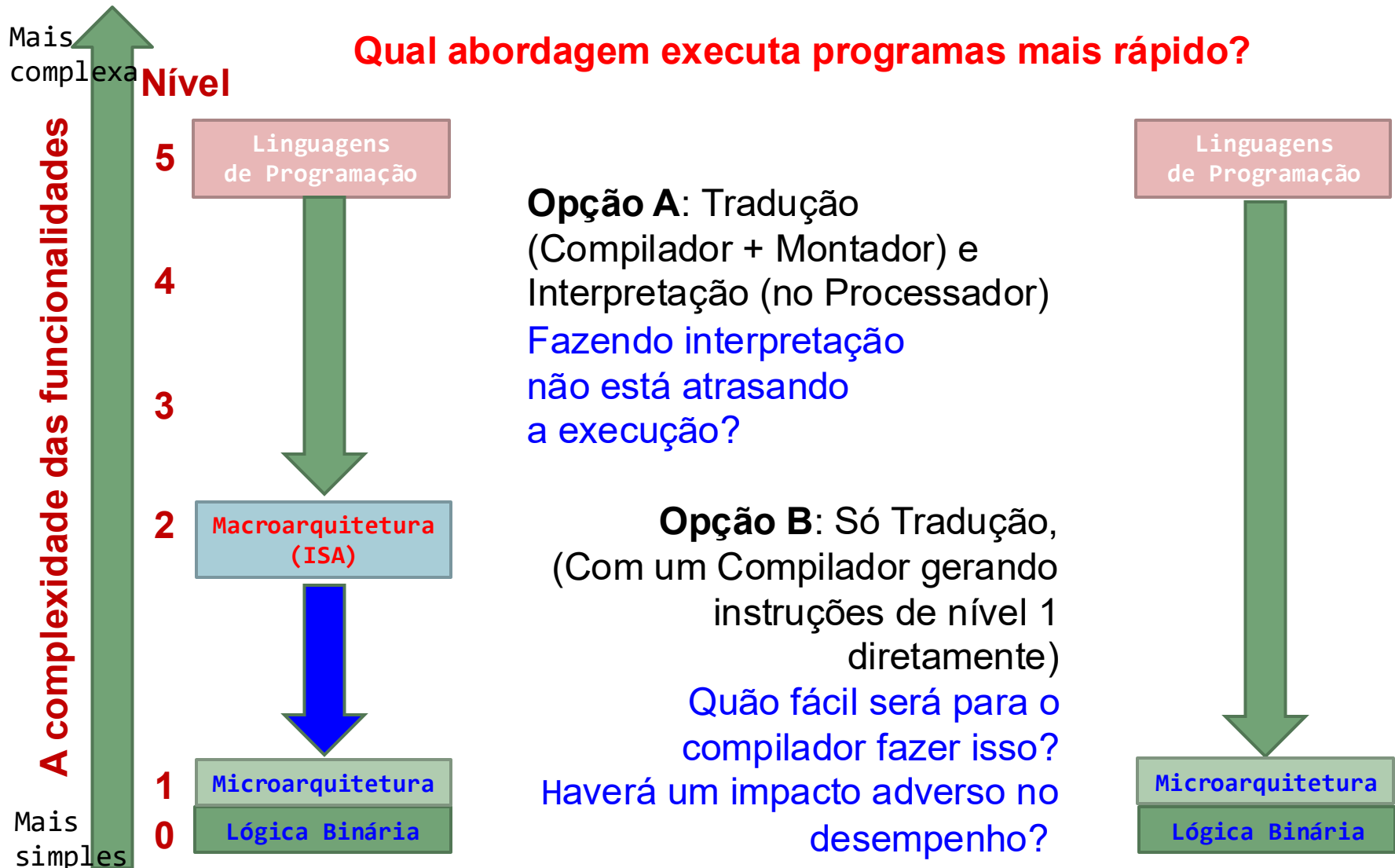
alu := tir; if n then goto



O Microprograma de MIC-1

- A memória de controle tem 256×32 bits, isto é 8K bits, este microprograma ocupa só $79 \times 32 = 2528$ bits
 - Mas não é a única implementação
 - Têm possibilidades para reduzir o tamanho da microinstrução sem perder funcionalidade ... (vai pensando nisso)
- Na "*média*" 9 ciclos (μ insts) para executar uma macroinstrução
- Identificar cada macroinstrução tem um *custo* significativo
 - Será que tem uma maneira para reduzir este custo?
 - Em hardware, é muito provável. Como programador, você faria o que?
- MIC-1 (ainda) não é uma implementação otimizada em termos de desempenho e custo

Tópico 2: A grande pergunta



Se $(i \leq n)$ então $m := a[i];$

- Você tem o papel de ser um compilador
- (Opção A) Traduz diretamente para uma sequência de *macroinstruções* correspondentes
 - Qual é o desempenho e custo do macroprograma?

Nossa ISA

ISA MAC-1 (nível 2)

- 23 Macroinstruções
- de 16 bits (coluna 1)
- Opcode crescente (4 ou 7 bits), não de tamanho fixo
- Três tipos de endereçamento:
 - Endereçamento direto
 - Endereçamento imediato
 - Endereçamento indexado
- Um operando de 12 bits ou 8 bits ou sem operando
 - Não determina diretamente o número de dados necessário para a operação

	Binary	Mnemonic	Instruction	Meaning
0000	xxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001	xxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010	xxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011	xxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100	xxxxxxxxxxx	JPOS	Jump positive	If $ac \geq 0$ then $pc := x$
0101	xxxxxxxxxxx	JZER	Jump zero	If $ac = 0$ then $pc := x$
0110	xxxxxxxxxxx	JUMP	Jump	$pc := x$
0111	xxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000	xxxxxxxxxxx	LODL	Load local	$ac := m[sp + x]$
1001	xxxxxxxxxxx	STOL	Store local	$m[x + sp] := ac$
1010	xxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp + x]$
1011	xxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp + x]$
1100	xxxxxxxxxxx	JNEG	Jump negative	If $ac < 0$ then $pc := x$
1101	xxxxxxxxxxx	JNZE	Jump nonzero	If $ac \neq 0$ then $pc := x$
1110	xxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1; m[sp] := pc; pc := x$
1111000	00000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001	00000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010	00000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011	00000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100	00000000	RETN	Return	$pc := m[sp]; sp := sp + 1$
1111101	00000000	SWAP	Swap ac, sp	$tmp := ac; ac := sp; sp := tmp$
1111110	yyyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
1111111	yyyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

xxxxxxxxxxx is a 12-bit machine address; in column 4 it is called x .
 yyyyyyy is an 8-bit constant; in column 4 it is called y .

Se $(i \leq n)$ então $m := a[i]$;

- Traduz diretamente para uma sequência de **macroinstruções** correspondentes:

- Duas de várias implementações:

- Analisando a 1ª

- Custo: 8×16 bits

- Desempenho:

- $(3 \text{ ou } 8) \times \sim 9$ ciclos

- 26 ou $(27 + 47)$ ciclos

LODD &n
SUBD &i
JNEG saída

LODD &i
SUBD &n
JZER dentro
JPOS saída

LOCO &a
ADDD &i
PSHI
POP
STOD &m

saída: ...

dentro: LOCO &a
ADDD &i
PSHI
POP
STOD &m

saída: ...

Se $(i \leq n)$ então $m := a[i]$;

- Você tem o papel de ser um compilador de novo
- (Opção B) Traduz diretamente para um conjunto de *microinstruções* correspondentes
 - Qual é o desempenho e custo do microprograma?

Se $(i \leq n)$ então $m := a[i]$;

- Você tem o papel de ser um compilador de novo
- (Opção B) Traduz diretamente para um conjunto de *microinstruções* correspondentes
 - Qual é o desempenho e custo do microprograma?
- Este é um exercício puramente hipotético
 - A arquitetura MIC-1, como está, não suporta este estilo de execução
 - Este exercício é só para validar a hipótese de Von Neumann

O Microprograma de MIC-1

- Este microprograma tem 79 microinstruções de 32 bits

0: mar:=pc; rd;	Busca	40: tir:=lshift(tir); if n then goto 46;	
1: pc:=pc + 1; rd;	Busca	41: alu:=tir; if n then goto 44;	
2: ir:=mbr; if n then goto 28;	Busca/Identifica	42: alu:=ac; if n then goto 22;	{1100 = JNEG}
3: tir:=lshift(ir + ir); if n then goto 19;	Identifica	43: goto 0;	
4: tir:=lshift(tir); if n then goto 11;	Identifica	44: alu:=ac; if z then goto 0;	{1101 = JNZE}
5: alu:=tir; if n then goto 9;	Identifica	45: pc:=band(ir.amask); goto 0;	
6: mar:=ir; rd;	Executa	46: tir:=lshift(tir); if n then goto 50;	
	{0000 = LODD}	47: sp:=sp + (-1);	{1110 = CALL}
7: rd;	Executa	48: mar:=sp; mbr:=pc; wr;	
8: ac:=mbr; goto 0;	Executa	49: pc:=band(ir.amask); wr; goto 0;	
9: mar:=ir; mbr:=ac; wr;	{0001 = STOD}	50: tir:=lshift(tir); if n then goto 65;	
10: wr; goto 0;	Executa	51: tir:=lshift(tir); if n then goto 59;	
11: alu:=tir; if n then goto 15;	Identifica	52: alu:=tir; if n then goto 56;	
12: mar:=ir; rd;	{0010 = ADDD}	53: mar:=ac; rd;	{1111-0000 = PSHI}
13: rd;	Executa	54: sp:=sp + (-1); rd;	
14: ac:=mbr + ac; goto 0;	Executa	55: mar:=sp; wr; goto 10;	
15: mar:=ir; rd;	{0011 = SUBD}	56: mar:=sp; sp:=sp + 1; rd;	{1111-0010 = POPI}
16: ac:=ac + 1; rd;	Executa	57: rd;	
17: a:=inv(mbr);	Executa	58: mar:=ac; wr; goto 10;	
18: ac:=ac + a; goto 0;	Executa	59: alu:=tir; if n then goto 62;	
19: tir:=lshift(tir); if n then goto 25;	Identifica	60: sp:=sp + (-1);	{1111-0100 = PUSH}
20: alu:=tir; if n then goto 23;		61: mar:=sp; mbr:=ac; wr; goto 10;	
21: alu:=ac; if n then goto 0;	{0100 = JPOS}	62: mar:=sp; sp:=sp + 1; rd;	{1111-0110 = POP}
22: pc:=band(ir.amask); goto 0;		63: rd;	
23: alu:=ac; if z then goto 22;	{0101 = JZER}	64: ac:=mbr; goto 0;	
24: goto 0;		65: tir:=lshift(tir); if n then goto 73;	
25: alu:=tir; if n then goto 27;		66: alu:=tir; if n then goto 70;	
26: pc:=band(ir.amask); goto 0;	{0110 = JUMP}	67: mar:=sp; sp:=sp + 1; rd;	{1111-1000 = RETN}
27: ac:=band(ir.amask); goto 0;	{0111 = LOCO}	68: rd;	
28: tir:=lshift(ir + ir); if n then goto 40;		69: pc:=mbr; goto 0;	
29: tir:=lshift(tir); if n then goto 35;		70: a:=ac;	{1111-1010 = SWAP}
30: alu:=tir; if n then goto 33;		71: ac:=sp;	
31: a:=ir + sp;	{1000 = LODL}	72: sp:=a; goto 0;	
32: mar:=a; rd; goto 7;		73: alu:=tir; if n then goto 76;	
33: a:=ir + sp;	{1001 = STOL}	74: a:=band(ir.smask);	{1111-1100 = INSP}
34: mar:=a; mbr:=ac; wr; goto 10;		75: sp:=sp + a; goto 0;	
35: alu:=tir; if n then goto 38;		76: a:=band(ir.smask);	{1111-1110 = DESP}
36: a:=ir + sp;	{1010 = ADDL}	77: a:=inv(a);	
37: mar:=a; rd; goto 13;		78: a:=a + 1; goto 75;	
38: a:=ir + sp;	{1011 = SUBL}		
39: mar:=a; rd; goto 16 ;			

Se $(i \leq n)$ então $m := a[i]$;

- Traduz diretamente para uma sequência de microinstruções correspondentes:

- **Desempenho:**

- 7 ou 14 ciclos **se usa a MC**

- 5 vezes melhor



- **Custo:** 14 × 32 bits

- ~4 vezes mais p/ 1 linha
 - Para um programa(s) inteiro?
 - μprograma só custa 316 bytes

saída: ...

```
mar := &n; rd      # busca n
rd;                # 2cicl p/ler
ac := mbr;         # ac recebe n
mar := &i; rd      # busca i
rd; ac := ac + 1   # subtração 1
a := inv(mbr);     # subtração 2
alu := ac + a; if n goto saída;
rd;                # MAR já tem &i
ac := &a; rd;      # carreg. end. a
ac := mbr + ac;    # end. a[i]
mar := ac; rd;     # busca a[i]
rd;
ac := mbr;
mar := &m; mbr := ac; wr;
wr;                # atualizar m
```